Transaction Correctness for SMS Style Wallet Requests Under Duplicate Delivery and Failure

Samuel Chukwuebuka Akabuike

Independent technical report | 29 October 2026 | Not peer reviewed

Abstract

This study revisits the transaction-correctness problem raised by a 2021 SMS-accessible wallet prototype. A separate 2026 Spring Boot and PostgreSQL prototype uses gateway message identifiers, transaction-scoped advisory locks, row locking, and atomic ledger writes. Six local synthetic trials used 10 or 30 concurrent requests per scenario. Across those trials, 120 baseline payments settled; 120 duplicate PAY deliveries produced six payment intents; and 120 competing CONFIRM deliveries settled each of six intents once. Six expired confirmations created no ledger entries, while six injected credit-ledger failures rolled back wallet updates. These observations support correctness for the tested schedules only. The experiment uses a simulated HTTP gateway and fictional balances; it does not measure carrier delivery, real-money operation, broad workload performance, or end-to-end security. Code and raw request results are available online.

Keywords: SMS-style transactions; idempotency; wallet ledger; fault injection; PostgreSQL

1 Introduction and historical source

The 2021 Federal University of Technology Owerri project explored payments initiated by SMS when the handset lacked mobile data. Its aim was stated plainly in the original report:

> *“The aim of this study is to develop an improved internet independent SMS-Based payment platform.”*
> *Akabuike and Victory, 2021, printed p. 15; reproduced unchanged in [2].*

The original prototype exposed Java API operations for enrolment, payment, balance enquiry, recharge, and PIN reset. It relied on a networked gateway and back end even though the handset could interact by SMS. Manual API checks demonstrated individual flows, but the report did not present a repeatable evaluation of duplicate delivery, concurrency, or transaction rollback. The present work asks a narrower question: can a newly built backend preserve balance and ledger correctness in selected synthetic duplicate, expiry, and database-failure schedules?

The 2021 report was jointly submitted by Samuel Chukwuebuka Akabuike and Philip Chimbuchi Victory. This report and the 2026 Spring Boot implementation are separate work by Samuel Chukwuebuka Akabuike. Historical passages below remain attributed to the jointly submitted source [1], and the archival selections can be inspected through [2].

2 Research question and correctness criteria

The central question is whether one payment intent can produce at most one settlement when gateway requests are repeated, while every committed transfer remains balanced. The experiment checks four observable criteria. First, repeated delivery of one message ID and identical body returns one stored response and creates one intent. Second, competing confirmation messages for one intent cause one debit and one credit only. Third, confirmation after expiry produces no transfer. Fourth, an exception during ledger writing leaves balances, ledger, and payment state unchanged. These are finite test claims, not a proof that every possible schedule is safe.

HTTP POST is not inherently idempotent; application-level identifiers are therefore necessary if a client may repeat a request after uncertainty about its outcome [3]. The implementation records a gateway message ID with a body hash and response. PostgreSQL transaction-scoped advisory locks serialize handling of the same ID before a record is read or inserted [4]. Payment-intent and wallet row locks protect settlement. A single database transaction writes the two wallet changes and balancing ledger entries; an unchecked database exception causes rollback [5].

3 Prototype and experimental method

The new code targets Java 21 and was exercised with OpenJDK 26.0.1, Spring Boot 4.0.9, and PostgreSQL 18.4 on one arm64 macOS host with 10 logical CPUs. One local application process received signed HTTP requests that simulated an SMS gateway. Accounts, telephone numbers, balances, and PINs were synthetic. No real SMS provider, bank, or money movement was used. The gateway HMAC checks the simulated transport request; it does not authenticate an SMS sender across the mobile network.

The harness ran three trials with 10 requests per concurrent scenario and three trials with 30.

Each trial created a fresh customer and merchant, seeded a fictional NGN 100,000 customer balance, and performed the following fixed sequence. Baseline PAY/CONFIRM pairs transferred NGN 1 per pair. Concurrent duplicate PAY deliveries used the same gateway message ID and body for a NGN 2 intent. Concurrent confirmations then used distinct delivery IDs for that intent. A further intent was forced past expiry in the test database before confirmation. Finally, a temporary PostgreSQL trigger rejected the CREDIT ledger row for another intent, after wallet updates had begun, and was removed immediately after the check. The harness asserted wallet balances, intent status, ledger row counts, and the sum of ledger deltas after each scenario.

The 2021 report described manual endpoint testing in its own words:

> *"before integration, each and everyone of this softwares had to be tested with the Postman software"*
> *Akabuike and Victory, 2021, printed p. 76; reproduced unchanged in [2].*

That historical test description is not the evidence for this paper. The 2026 result files contain each local HTTP status, response, and elapsed client time, plus trial summaries. Median and 95th-percentile values below are descriptive nearest-rank statistics over the recorded requests. They are not independent carrier latencies or confidence bounds. The code commit used for the runs is 19bcb88342b2a56e1fa70702d66246f73ff3ba7f.

4 Results

All six trials completed their programmed correctness assertions. The counts below combine the 10-request and 30-request workloads. An expected HTTP 500 occurred in each deliberately injected database-failure case; no such response occurred in the other recorded scenarios.

Scenario	Requests	Observed state	Ledger / balance result
Baseline PAY and CONFIRM	120 pairs	120 settlements	Two balanced rows per settled intent
Duplicate PAY, same ID	120	Six unique intents; all HTTP 200	No additional intents from repeats
Competing CONFIRM IDs	120	Six completed; 114 already completed	One settlement per intent
Expired confirmation	6	Six intents marked expired	Zero transfer rows
Injected CREDIT failure	6	Six expected HTTP 500 responses	Zero transfer rows; balances unchanged

Request times are local end-to-end HTTP observations. The confirm path includes bcrypt PIN verification, so it is not directly comparable with PAY. The difference between workload sizes cannot be interpreted as a scalability result because trial order, database warm-up, and background host activity were not controlled.

Local request type	Recorded requests	Median ms	P95 ms
Baseline PAY	30	5.84	8.41
Baseline CONFIRM	30	303.41	307.38
Duplicate PAY	30	9.37	20.34
Competing CONFIRM	30	309.81	323.83
Baseline PAY	90	4.71	7.12
Baseline CONFIRM	90	301.79	306.21
Duplicate PAY	90	8.18	13.93
Competing CONFIRM	90	310.64	317.55

5 Interpretation and limits

The tested duplicate deliveries did not produce duplicate intents or settlements, and the database trigger showed rollback of a partly executed settlement. The result is strongest for correctness of the specific single-process, PostgreSQL-backed schedules in the repository. It does not show that a real SMS exchange completes, that user-visible responses arrive, or that a failed operation can always be recovered without operator action. The injected exception returned HTTP 500; a production design would need a defined retry and customer-support path.

The distinction between one repeated message ID and two different IDs matters. Message-ID deduplication prevents replay of the same gateway delivery, but two independently identified PAY requests can still create two payment intents. The user must confirm an intent before funds move, yet this is not semantic deduplication of a human intention. A future study should model that ambiguity, multiple application instances, larger concurrent workloads, database restarts, and randomly generated schedules. It should also measure CPU, lock contention, throughput, and uncertainty across more independent runs before making performance claims.

Security remains a separate unresolved question. The prototype preserves PIN-bearing SMS commands from the historical interaction design; ordinary SMS is not end-to-end encrypted, and the gateway signature cannot establish that the handset owner authored a message. SIM-swap, sender spoofing, interception, replay with a new ID, logging exposure, and account recovery require a redesigned interaction and threat review. This prototype must not handle real funds or personal payment data.

The original report also documented a notification problem without rewriting its language:

> *"Designing the SMS API for message notification after a successful or failed transaction ran into a bug which caused our message credit to run down in thousands of Naira."*
> *Akabuike and Victory, 2021, printed p. 18; reproduced unchanged in [2].*

That observation concerns the 2021 prototype and was not retested here. It motivates future work on delivery acknowledgement and cost controls; neither is established by the current results.

6 Conclusion

A new implementation of the 2021 SMS-accessible wallet idea preserved balance and ledger invariants in six bounded local trials involving duplicate deliveries, competing confirmations, forced expiry, and injected database failure. The repository makes the code, test harness, and raw results inspectable. The evidence supports a narrow prototype correctness claim and provides a starting point for a larger study of delivery, recovery, security, and performance. It does not establish a deployable payment system.

Data and code availability

Source code, the experiment harness, aggregate summary, and raw request CSV files: https://github.com/Akasam79/sms-transaction-reliability-lab. Exact experimental code commit: 19bcb88342b2a56e1fa70702d66246f73ff3ba7f. The archival 2021 source selections and 2026 technical note: https://doi.org/10.5281/zenodo.23766609. All experimental accounts and balances are synthetic.

References

[1] S. C. Akabuike and P. C. Victory, Development of an Improved Internet Independent SMS-Based Payment Platform, B.Tech Computer Science final-year project report, Federal University of Technology Owerri, July 2021.

[2] S. C. Akabuike, PayKeen: Original 2021 Project Selections with an Author's 2026 Technical Note, Zenodo archival technical report, 2026. https://doi.org/10.5281/zenodo.23766609

[3] R. Fielding, M. Nottingham, and J. Reschke, HTTP Semantics, RFC 9110, 2022, sec. 9.2.2. https://www.rfc-editor.org/rfc/rfc9110.html

[4] PostgreSQL Global Development Group, Explicit Locking, PostgreSQL 18 Documentation, sec. 13.3.5, 2026. https://www.postgresql.org/docs/18/explicit-locking.html

[5] Spring Framework, Using @Transactional, Spring Framework Reference Documentation, 2026. https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html